

第10章 算法设计技巧

建议学时:5-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第1-9章)

🎯 本章学习目标

- 掌握四大范式:贪心、分治、动态规划、回溯,说出适用场景
- 理解"局部最优 $\neq$ 全局最优",会用反例验证贪心猜想
- 能独立实现活动安排与哈夫曼编码(复用第6章 `heapq`)
- 掌握 DP 两大要素(最优子结构、重叠子问题),独立写出 LCS 与 0-1 背包
- 掌握随机化:洗牌与 `quickselect`,完成 `rand()` $\rightarrow$ `random` 迁移
- 掌握回溯模板"选择-递归-撤销",独立实现八皇后

💡 从 C 到 Python:本章主题如何迁移

前九章学"给定结构,分析复杂度";本章反过来:给定问题,自己设计算法。四大范式是四套"解题姿势":见什么问题套什么姿势。

C 程序员的习惯是"两层循环 + 标志位"式穷举:n 小时没问题,一到 20, $O(2^n)$ 就彻底失败。四种范式改造穷举:贪心剪掉次优分支,分治切出独立小块,DP 缓存重复计算,回溯按需展开搜索。

Python 给本章三样武器:① 闭包与生成器——C 的回调要函数指针加 `void*` (第7章 `qsort`),Python 用嵌套函数捕获外层变量,生成器 `yield` 惰性产出全部排列;② `random` 模块——`rand() % n` 的周期短、概率不均、全局状态三大问题全解决, `random.seed()` 固定种子;③ `random.shuffle`、`sorted`:手写版懂原理,标准库版做工程。

📦 前置知识自查

数学前置:递推式(第2章)、对数、均匀分布直觉。

C 语言前置:`malloc` 二维数组、函数指针(`qsort` 比较器)、递归函数。

依赖的前面章节:第2章(大 O 与分治复杂度)、第6章(二叉堆,哈夫曼用)、第7章(快排分割,`quickselect` 用)。

🚀 本章学习路线

1. 第一阶段(1.5 小时):读 §10.1-§10.2,手写活动安排与哈夫曼建树
2. 第二阶段(1.5 小时):读 §10.3,画 LCS 的 DP 表,默写背包
3. 第三阶段(1.5 小时):读 §10.4-§10.5,验证洗牌均匀性,实现八皇后
4. 第四阶段(实验):完成章末综合实验:背包双版本 + 八皇后 + 计时对比
5. 验收标准:陌生问题 1 分钟内说出"范式、状态、复杂度"

本章知识导图

- §10.1 贪心:活动安排、哈夫曼、硬币反例
- §10.2 分治:最近点对与分治三要素
- §10.3 动态规划:重叠子问题与状态设计(记忆化、LCS、背包)
- §10.4 随机化:洗牌与第 k 小(Fisher-Yates、quickselect)
- §10.5 回溯:选择-递归-撤销(八皇后、子集和)
- §10.6 四大范式对比与选型
- §10.7 工程实践:DP 检查清单与随机化测试

§10.1 贪心:活动安排与哈夫曼编码

10.1.1 贪心思想:每步取局部最优

贪心(greedy) 一句话: 每一步选"当前看起来最好"的选择,希望最终全局最优。它不回头、不后悔,因此要求"无后效性"。经典案例:找零钱、活动安排、哈夫曼。注意: 贪心正确性必须逐题验证,没有通用模板——这是它最大的坑(误区1)。

10.1.2 活动安排问题

会议室只有一个,每个活动有开始与结束时间,同一时刻只能进行一个,问最多能安排几个?策略: 按结束时间最早排序,依次选不冲突的活动——**最早结束优先**。例:10 个活动排序后扫描,选中 (1,4)、(5,7)、(8,11),恰为最优。

示例 10-1 活动安排:最早结束优先

```
def activity_select(acts):
    """活动安排:按结束时间升序,依次选不冲突的活动"""
    acts.sort(key=lambda a: a[1])      # 贪心策略:最早结束优先
    res = []
    last_end = -1
    for start, end in acts:
        if start >= last_end:           # 不冲突:直接选
            res.append((start, end))
            last_end = end
    return res
```

10.1.3 为什么"最早结束"是对的

为什么不用"最早开始"或"持续时间最短"?直觉: 结束得越早,留给后面的空闲越多。严格证明叫**交换论证**(教材给出):把第一个活动换成"结束最早的活动"不会变差——它结束得不晚,后面所有活动仍放得下;逐步替换即可。注意直觉不是证明,后文有反例。

10.1.4 任务调度简介

同族另一个贪心:**单机任务调度**——已知各任务处理时间,使**平均完成时间最小**。策略: **最短处理时间优先(SJF)**:短任务先完成,大家等待的总时间最小;交换论证同样成立。这是操作系统"短作业优先"的原型。复杂度 $O(n \log n)$ 。

10.1.5 哈夫曼编码:前缀码与二叉堆

变长编码:高频字符用短码、低频用长码。但变长有歧义: $a=0, b=01$ 时, 01 是 $a+b$ 还是 b ?靠**前缀码**解决: 任何编码都不是另一个编码的前缀。哈夫曼算法构造**平均码长最短**的前缀码:每次合并频次最小的两棵(频次相加),直到剩一棵;频次越低,离根越远、码越长。取最小用第6章的 `heapq`:

示例 10-2 哈夫曼编码:heapq 建树与编码

```
import heapq

class HNode:
    """哈夫曼结点:叶存字符,内部结点 ch 为 None"""
    def __init__(self, ch, freq):
        self.ch = ch
        self.freq = freq
        self.left = self.right = None

    def is_leaf(self):
        return self.ch is not None

def build_huffman(freq):
    # freq: {字符: 频次}
    pq = []
    # heapq 默认小顶堆,正好符合要求
    for i, (ch, f) in enumerate(freq.items()):
        heapq.heappush(pq, (f, i, HNode(ch, f)))
    nxt = len(pq)
    while len(pq) > 1:
        # 合并频次最小的两棵
        f1, _, a = heapq.heappop(pq)
        f2, _, b = heapq.heappop(pq)
        p = HNode(None, f1 + f2) # 内部结点,频次相加
        p.left, p.right = a, b
        heapq.heappush(pq, (f1 + f2, nxt, p)) # 序号破平局,避免比较结点
        nxt += 1
    return pq[0][2]

def gen_codes(root, path="", codes=None):
    # DFS:左 0 右 1,叶即编码
    if codes is None:
        codes = {}
    if root.is_leaf():
        codes[root.ch] = path
    else:
        gen_codes(root.left, path + "0", codes)
        gen_codes(root.right, path + "1", codes)
    return codes
```

C 的写法(手写最小堆)	Python 的写法(heapq)
数组 + 上滤下滤手写约 60 行	heapq 自带小顶堆,几行完成
每次取最小线性扫描,建树 $O(n^2)$	堆取最小 $O(\log n)$,建树 $O(n \log n)$
比较逻辑写死在代码里	元组 (频次, 序号, 结点) 天然可比,免写比较器
结点手动 <code>free</code> ,易泄漏	对象由引用计数自动回收

重点:为什么前缀码无歧义。哈夫曼编码是**树上的叶编码**:字符对应叶,编码是根到叶的路径。根到叶的路径不经过另一棵叶,任意两叶编码互不为前缀——解码器从左到右扫描,到叶就输出一个字符,绝不卡壳。字符必须全放叶上,放进内部结

点就产生歧义。

⚠ 误区 1:贪心不是万能——硬币反例

硬币面值 {1, 3, 4} 凑 6 元。"尽量用大面值"给出 4+1+1 共 3 枚;最优是 3+3 共 2 枚。每步局部最优(先用 4 元)恰恰堵死了全局最优(两个 3 元)。贪心无通用证明:先想反例,再考虑交换论证;第9章 Dijkstra、Kruskal 敢用贪心,是因有严格证明。

§10.2 分治:最近点对与分治三要素

10.2.1 回顾:分治三要素与复杂度

分治在第2、7章反复出场(最大子序列和、归并、快排),归纳三要素:分(Divide)——切成更小的子问题;治(Conquer)——递归求解;合(Combine)——合并答案。归并与最大子序列和都是 $T(n) = 2T(n/2) + O(n)$:每层 $O(n)$ 、共 $\log_2 n$ 层,总计 $O(n \log n)$;另一形态 $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$ (二分)。遇到 $T(n) = aT(n/b) + f(n)$,画递归树看每层总代价。

🚀 记号说明(沿用第2章)

递推式用行内代码: $T(n) = 2T(n/2) + O(n)$ 表示"两个规模 $n/2$ 的子问题 + 线性合并"; n^2 、 2^n 、 $\log_2 n$ 同第2章;"平均复杂度"指输入等概率分布下的期望值。

10.2.2 最近点对:思想与结论

平面上 n 个点,求距离最近的一对。暴力两两枚举 $O(n^2)$ 。分治:① **分**:按 x 排序从中间切两半;② **治**:递归求两半各自最小距离得 d ;③ **合**(精髓):跨中线的点对 只需检查"距中线 d 以内"的条带——按 y 排序后每点至多与后面 7 个点比较 ($d \times 2d$ 矩形内至多塞下常数个点,证明见教材)。总复杂度 $O(n \log n)$ 。

示例 10-3 最近点对:分治骨架

```
import math

def dist(a, b):
    # 点用二元组 (x, y) 表示
    return math.hypot(a[0] - b[0], a[1] - b[1])

def closest_pair(pts):
    # pts 已按 x 升序;返回最小距离
    n = len(pts)
    if n <= 3:
        # 基准情形:暴力
        return min(dist(pts[i], pts[j])
                    for i in range(n) for j in range(i + 1, n))
    mid = n // 2
    mid_x = pts[mid][0]
    d = min(closest_pair(pts[:mid]), closest_pair(pts[mid:]))
    strip = [p for p in pts if abs(p[0] - mid_x) < d]  # 合:距中线 d 内
    strip.sort(key=lambda p: p[1])                    # 按 y 排序
    for i in range(len(strip)):
        for j in range(i + 1, len(strip)):
            if strip[j][1] - strip[i][1] >= d:
                break
            # y 差已 ≥ d,后面更远
            d = min(d, dist(strip[i], strip[j]))
    return d
```

§10.3 动态规划:重叠子问题与状态设计

10.3.1 分治与 DP:重叠子问题

分治假设子问题**互不相交**。但递归分解常产生**大量相同子问题**:fib(5) 要算 fib(4) 与 fib(3),fib(4) 又要算 fib(3)——fib(3) 被重复算。**动态规划(DP)**:把子问题答案存进表格,用到查表,每个子问题只算一次。分治靠递归树,DP 靠表格,这是本质区别。

10.3.2 斐波那契:记忆化与自底向上

第1章的朴素递归是 $O(2^n)$,fib(50) 算到地老天荒。两种救法:**记忆化**(自顶向下:递归 + 查表)与**自底向上**(从 fib(0) 按序填表),都做到 $O(n)$;后者省掉递归开销,工程首选。

示例 10-4 斐波那契三版

```
def fib_rec(n):          # 朴素递归  $O(2^n)$ (第1章)
    return n if n <= 1 else fib_rec(n - 1) + fib_rec(n - 2)

def fib_memo(n, memo=None): # 记忆化(自顶向下):先查表再递归
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    memo[n] = n if n <= 1 else fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

def fib_iter(n):          # 自底向上:从小到大填表, $O(n)$ 
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

10.3.3 最长公共子序列 LCS

子序列:按顺序挑出(可跳着)的字符序列。**LCS**:两字符串的最长公共子序列。状态: $dp[i][j]$ = s_1 前 i 个与 s_2 前 j 个的 LCS 长度。转移:① 字符相等 $dp[i][j] = dp[i-1][j-1] + 1$;② 否则取 $\max(dp[i-1][j], dp[i][j-1])$ 。填完表从右下角沿来路逆推还原出具体 LCS——**方案回溯**是 DP 常客。例:ABCBDAB 与 BDCABA 得长度 4,LCS 为 "BCBA"(不唯一)。

示例 10-5 LCS:DP 表构造与回溯

```

def lcs_len(s1, s2):
    """返回 LCS 长度并填好 dp 表"""
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1          # 相等:左上+1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) # 不等:取较大
    return dp

def lcs_trace(s1, s2, dp):
    """回溯:从右下角沿来路逆推,收集 LCS 字符"""
    res = []
    i, j = len(s1), len(s2)
    while i > 0 and j > 0:
        if s1[i - 1] == s2[j - 1]:
            res.append(s1[i - 1])                        # 相等:属于 LCS
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1                                        # 从上方来
        else:
            j -= 1                                        # 从左方来
    return "".join(reversed(res))

```

10.3.4 0-1 背包:二维表与一维优化

0-1 背包: n 件物品,第 i 件重 $w[i]$ 、值 $v[i]$,容量 W ,每件要么整放要么不放,求最大总价值。状态: $dp[i][c]$ = 只用前 i 件、容量 c 的最大价值。转移:放第 i 件得 $dp[i-1][c-w[i-1]] + v[i-1]$,不放得 $dp[i-1][c]$,取最大。二维表 $O(nW)$ 时间空间;一维滚动数组把空间压到 $O(W)$,代价是容量必须逆序循环——正序时 $dp[c-w]$ 已被本轮更新,同一件被放多次(误区 2)。

示例 10-6 0-1 背包:二维与一维

```
def knap2d(w, v, W):
    """二维版:dp[i][c] = 只用前 i 件、容量 c 的最大价值"""
    n = len(w)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for c in range(1, W + 1):
            if c < w[i - 1]:
                dp[i][c] = dp[i - 1][c]
            else:
                dp[i][c] = max(dp[i - 1][c],
                               dp[i - 1][c - w[i - 1]] + v[i - 1])
    return dp[n][W]

def knap1d(w, v, W):
    """一维版:滚动数组,容量必须逆序!正序变成完全背包"""
    dp = [0] * (W + 1)
    for i in range(len(w)):
        for c in range(W, w[i] - 1, -1): # 逆序是关键
            dp[c] = max(dp[c], dp[c - w[i]] + v[i])
    return dp[W]
```

C 的写法(手工二维数组)	Python 的写法(list 列表)
malloc 每行再逐行 free,易错	<code>[[0] * (W+1) for _ in range(n+1)]</code> 一条语句建表
越界读写是未定义行为	越界抛 <code>IndexError</code> ;调试友好
维度要手把手传参	<code>len(dp)</code> 自带维度,切片与推导式可遍历
换一维滚动数组要重写全部下标	两版结构一致,只差循环方向

⚠ 误区 2:一维背包的容量循环写成正序

一维版要求 `dp[c - w[i]]` 还是"上一轮"的值。逆序保证更新 `dp[c]` 时 `dp[c - w[i]]` 尚未被本轮更新;正序则可能已被更新,第 i 件被用多次——正是完全背包的转移式。口诀:01 背包逆序,完全背包正序。验证:小样例手工跑,或与二维版对拍(实验任务 3)。

★ 重点:DP 解题五步(全章最重要的清单)

① **定义状态**:dp 下标代表什么?要完整描述局面且不冗余;② **写转移方程**:当前状态由哪些更小的状态转移来?③ **定初值与边界**:首行首列、空输入、容量 0 的答案?④ **定计算顺序**:依赖项先算好(通常从小到大);⑤ **方案回溯**:需要方案时记录选择或从终态逆推。

§10.4 随机化:洗牌与第 k 小

10.4.1 Fisher-Yates 洗牌

把数组随机打乱,朴素做法(任意位置交换)分布不均匀。Fisher-Yates 的关键:第 i 步只与 $[0, i]$ 中的随机位置交换(从后往前),交换后位置 i 被锁定。共 $n!$ 种可能交换序列,与排列——对应(`random.shuffle` 就是它)。

示例 10-7 Fisher-Yates 与 `random.shuffle` 并排

```
import random

def fisher_yates(a):
    """手写 Fisher-Yates:从后往前,每位置与 [0, i] 中随机位置交换"""
    for i in range(len(a) - 1, 0, -1):
        j = random.randint(0, i)      # 关键:上界是 i!
        a[i], a[j] = a[j], a[i]
    return a

def shuffle_std(a):
    """标准库写法:一行完成,内部就是 Fisher-Yates"""
    random.shuffle(a)
    return a
```

⚠ 误区 3:洗牌上界取 $[0, n-1]$

若每步从 n 个位置随机挑,共 n^n 种等可能序列,排列只有 $n!$ 种—— n^n 不被 $n!$ 整除,必有不均:如 $n=3$ 时 $(1,2,3)$ 概率 $2/9$ 。Fisher-Yates 每步可选数递减($1 \cdot 2 \cdot \dots \cdot n = n!$),——对应。实证:洗 10 万次统计排列分布。

10.4.2 quickselect:求第 k 小

求第 k 小(0 基)。排序后取第 k 位要 $O(n \log n)$ 。**quickselect** 复用第7章快排的**分割(partition)**:枢轴被放到最终位置 p ; $p == k$ 直接返回,否则只递归"包含第 k 小"的那一侧。期望代价 $n + n/2 + n/4 + \dots = 2n$,平均 $O(n)$;最坏(每次切最边上) $O(n^2)$ 。

示例 10-8 quickselect 与 sorted 对照

```
def partition(a, lo, hi):
    """快排分割:枢轴放到最终位置,返回下标"""
    pivot = a[hi]                # 简化:取末元素做枢轴
    i = lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i

def quick_select(a, k):
    """第 k 小(0 基):分割后只处理"包含第 k 小"的那一侧"""
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        p = partition(a, lo, hi)
        if p == k:
            return a[k]
        if p > k:
            hi = p - 1
        else:
            lo = p + 1
    return -1

# 标准库对照:sorted(a)[k] 一行求第 k 小,但代价是  $O(n \log n)$ 
```


10.4.3 随机数:从 C 的 rand() 到现代随机库

C 的 `rand()/srand()` 三个硬伤: ① 周期短(约 2^{31}); ② `rand() % n` 概率不均—— n 不整除 `RAND_MAX` 时余数偏斜, 低位周期短; ③ 全局状态, 线程不安全、不可复制。Python `random` 模块基于梅森旋转 MT19937, 周期 $2^{19937}-1$; `random.seed(seed)` 固定种子则序列可复现, 是复现性的基石; 引擎与分布不必手动管理, 全局唯一, 直接调用即可。`randint(a, b)` 是闭区间 $[a, b]$ 和 C 的 $[0, n)$ 习惯不同, 注意别多 1。

C 的写法(rand 家族)	Python 的写法(random 模块)
<code>srand(time(NULL)); rand();</code>	<code>random.seed(2026); random.random()</code>
<code>rand() % n</code> 概率不均	<code>random.randint(a, b)</code> 闭区间均匀整数
全局状态, 线程不安全, 不可复制	全局单例, 自动播种, 线程安全有保证
周期约 2^{31} , 质量差	MT19937 周期 $2^{19937}-1$, 通过统计检验
洗牌/抽样要自己写	<code>random.shuffle</code> / <code>random.sample</code> 一行完成

10.4.4 随机算法的工程观

quickselect 平均 $O(n)$ 、最坏 $O(n^2)$ ——“数学概率上快”的含义: 把最坏情形的概率压到工程上可忽略。随机选枢轴后, 能触发最坏的输入几乎无法被构造; 对单次最坏延迟有硬性要求(实时系统)则改用确定性算法。

§10.5 回溯:选择-递归-撤销

10.5.1 回溯模板:选择-递归-撤销

当搜索空间是“一步步做决定”的结构时, 回溯(backtracking) 是穷举的优雅写法, 核心三行——① 选择: 决定写进状态; ② 递归: 进入下一层; ③ 撤销: 恢复状态, 尝试下一个选择。撤销是灵魂, 忘了它状态被污染。回溯的本质是带剪枝的深度优先穷举: 排列空间 $n!$, 子集空间 2^n ; 八皇后不剪枝要试 64 选 8, 剪枝后只访问约 1.5 万结点。剪枝两类: 可行性剪枝(不可能通向合法解)与最优性剪枝(不可能优于已知最好解)——回溯“慢但保证对”, 贪心“快但可能错”。Python 里递归函数用嵌套函数(闭包)捕获外层变量; 枚举全排列用生成器 `yield`, 逐个产出不占内存:

示例 10-9 全排列:yield 生成器

```
def perms(a):
    """yield 版全排列: 每层选一个元素, 惰性产出每种排列"""
    if len(a) <= 1:
        yield a[:]
        return
    for i in range(len(a)):
        rest = a[:i] + a[i + 1:]          # 选第 i 个, 剩余部分递归
        for p in perms(rest):
            yield [a[i]] + p              # 产出一种排列

# 用法: for p in perms([1, 2, 3]): print(p)    # 共 6 行
# 标准库对照: itertools.permutations([1, 2, 3]) 也是生成器, 按字典序
# 手写版懂原理(yield 的惰性), 标准库版做工程
```

10.5.2 八皇后:完整实现

在 $n \times n$ 棋盘放 n 个皇后,任意两个不能同行、同列、同对角线。暴力枚举 $C(64,8)$ 是天文数字;回溯逐行放皇后, $O(1)$ 冲突检查:列 $col[c]$;主对角线 $d1[row+c]$ (同一主对角线上 $row+c$ 相同);副对角线 $d2[row-c+n-1]$ ($row-c$ 相同,加 $n-1$ 防负下标)。放不下就回退换列——"回溯"由此得名。Python 版把"搜索"与"结果处理"解耦:闭包 dfs 收集到列表,入解必须 `pos[:]` 拷贝,否则后续撤销会污染已收集的解。

示例 10-10 八皇后:闭包与冲突检查

```
def n_queens(n):
    """八皇后:返回全部解的列号列表(每行一个皇后所在列)"""
    pos = [0] * n
    col = [False] * n
    d1 = [False] * (2 * n - 1)
    d2 = [False] * (2 * n - 1)
    sols = []

    def dfs(row):
        if row == n:
            sols.append(pos[:])          # 收集解(拷贝,防撤销污染)
            return
        for c in range(n):
            if col[c] or d1[row + c] or d2[row - c + n - 1]:
                continue                # 冲突剪枝
            pos[row] = c
            col[c] = d1[row + c] = d2[row - c + n - 1] = True  # 选择
            dfs(row + 1)                 # 递归
            col[c] = d1[row + c] = d2[row - c + n - 1] = False # 撤销

    dfs(0)
    return sols

# 结果:len(n_queens(8)) == 92;前 3 解(1 基):(1,5,8,6,3,7,2,4)、
# (1,6,8,3,7,4,2,5)、(1,7,4,6,8,2,5,3)
```

10.5.3 子集和问题

给定正整数集合与 target,能否选出若干元素恰好和为 target?每件"选/不选"两个分支,走到叶判断命中。加**可行性剪枝**:已超 target 直接剪掉;总和都不够则不进搜索。子集和没有贪心可依赖(硬币反例同款),只能搜索——这就是范式选型的意义。

§10.6 四大范式对比与选型

10.6.1 四范式大表

范式	核心特征	适用场景	典型例题
贪心	每步局部最优;无后效性;正确性需证明	能构造策略且验证正确	活动安排、哈夫曼、Dijkstra
分治	子问题互不相交;分-治-合;递推式	能切成独立子问题	归并、快排、最近点对
动态规划	子问题重叠;最优子结构;状态与转移	有重叠子问题与最优子结构	斐波那契、LCS、0-1 背包
回溯	DFS 搜索;选择-递归-撤销;剪枝	无多项式思路,规模 ≤ 20	八皇后、子集和、数独

10.6.2 选型决策

- **先看规模**: $n \leq 20 \rightarrow$ 回溯; n 达 $10^6 \rightarrow$ 多项式算法
- **有贪心猜想?** 策略可构造且反例想不出 $\rightarrow$ 试贪心并证明; 有反例 $\rightarrow$ 放弃
- **能切成独立子问题?** 左右互不影响 $\rightarrow$ 分治
- **有重复子问题?** $\rightarrow$ DP: 定义状态、写转移、填表
- **都不行?** $\rightarrow$ 回溯 + 剪枝, 或近似/启发式

10.6.3 什么时候"想不出多项式算法": NP 问题的一句话

旅行商、哈密顿回路等问题至今**没人找到**多项式时间算法, 普遍相信不存在——通常称 **NP 难**。遇到它们, 心态不是"我再想想", 而是: 规模小用回溯 + 剪枝, 规模大用近似或启发式, 并明确告诉用户这是近似解。

§10.7 工程实践: 状态检查、剪枝与随机化测试

10.7.1 DP 状态设计检查清单

- 状态参数能否完整描述"当前局面"? 少一个参数转移就丢信息
- 转移方程引用的状态都"更小"吗? (防循环依赖)
- 初始值与边界是否齐全? (首行首列、空输入、容量 0)
- 计算顺序是否保证依赖项先算好? (背包容量、LCS 行列)
- 空间能否优化? 二维表能否滚成一维?
- 需要输出方案吗? 要就保留完整表或记录选择

10.7.2 回溯剪枝的顺序

剪枝顺序影响常数: ① 可行性剪枝放最前 (冲突检查、超目标)——砍掉的子树最多; ② 对称性剪枝 (八皇后旋转/镜像省约 7/8 搜索, 选做); ③ 最优性剪枝最后做 (求最优解时用)。优先搜索分支更少的方向 (如子集和先放大元素), 也能显著加速。

10.7.3 随机化在测试中的应用

两个利器: ① **随机数据 + 对拍**——用 `random.seed()` 生成随机小样例, 把"手写实现"与"暴力实现"逐一比对, 十万个随机样例比十个手写边界更早暴露 bug (实验任务 3); ② **固定种子**——回归可复现, 换种子扩覆盖。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
活动安排(最早结束优先)	排序 + 线性扫描	$O(n \log n)$	$O(n \log n)$
任务调度(最短处理时间优先)	排序 + 线性扫描	$O(n \log n)$	$O(n \log n)$
哈夫曼建树	二叉堆(heapq)	$O(n \log n)$	$O(n \log n)$
最近点对	分治(条带合并)	$O(n \log n)$	$O(n \log n)$
最大子序列和	分治(第2章算法三)	$O(n \log n)$	$O(n \log n)$
归并排序	分治	$O(n \log n)$	$O(n \log n)$
快速排序	分治 + 随机化枢轴	$O(n \log n)$	$O(n^2)$
斐波那契	朴素递归	$O(2^n)$	$O(2^n)$
斐波那契	记忆化 / 自底向上	$O(n)$	$O(n)$
LCS	动态规划	$O(mn)$	$O(mn)$
0-1 背包	动态规划(二维或一维)	$O(nW)$	$O(nW)$
洗牌	Fisher-Yates / random.shuffle	$O(n)$	$O(n)$
第 k 小	quickselect / sorted	$O(n)$	$O(n^2)$
八皇后	回溯(冲突剪枝)	上界 $O(n!)$	上界 $O(n!)$
子集和	回溯(可行性剪枝)	$O(2^n)$	$O(2^n)$

最小必会清单

- 能说出贪心思想与"无后效性",举出活动安排、哈夫曼案例与硬币反例
- 能默写活动安排,说出复杂度 $O(n \log n)$
- 能默写哈夫曼建树(heapq 最小堆),解释前缀码解码无歧义
- 能说出分治三要素与最近点对合并思路,写出 $T(n) = 2T(n/2) + O(n)$
- 能说出分治与 DP 的本质区别(子问题独立 vs 重叠)与记忆化、自底向上
- 能独立写出 LCS 的 DP 表与回溯代码,解释 $dp[i][j]$ 的含义
- 能默写 0-1 背包两版,解释一维为何必须逆序
- 能默写 Fisher-Yates,说出"上界必须是 i ";会用 quickselect
- 能默写回溯模板"选择-递归-撤销",独立实现八皇后
- 会用 random.seed / random.randint / random.shuffle / sorted

自测题

Q 1. 活动安排为何按"结束时间最早"贪心?"开始时间最早"行吗?

✓ 参考答案

结束得越早,给后面留下的时间越多;交换论证保证任何最优解都能换成贪心解。按"最早开始"不行:活动 A(1,10) 霸占全天,(2,3)(4,5)(6,7) 全被挡住——贪心得 1 个,最优 3 个。

Q 2. 面值 {1,3,4} 凑 6 元,贪心给出什么?最优呢?说明什么?

✓ 参考答案

贪心 4+1+1 共 3 枚,最优 3+3 共 2 枚。局部最优 $\neq$ 全局最优:先用 4 元堵死了两个 3 元。贪心正确性必须逐题验证;人民币面值贪心成立,是"设计面值"的结果。

Q 3. 分治与动态规划的本质区别是什么?各举一个代表问题。

✓ 参考答案

分治的子问题互不相交(归并左半右半无重叠),复杂度来自递归树每层合并代价;DP 的子问题大量重叠(fib(3) 被重复计算),复杂度来自"每个状态只算一次"。代表:分治——归并;DP——LCS、0-1 背包。

Q 4. 0-1 背包的一维滚动数组为什么必须逆序?正序会怎样?

✓ 参考答案

逆序保证更新 $dp[c]$ 时 $dp[c-w[i]]$ 还是"没考虑第 i 件"的旧值;正序时它可能已被本轮更新,同一件物品被用多次,转移式变成完全背包。口诀:01 背包逆序,完全背包正序。验证:与二维版对拍(实验任务 3)。

Q 5. Fisher-Yates 每步的随机上界为什么必须是 i ?取 $[0, n-1]$ 会怎样?

✓ 参考答案

Fisher-Yates 每步可选数递减,共 $n!$ 种等可能交换序列,与排列一一对应,保证均匀。若每步从 n 个位置挑,共 n^n 种序列而排列只有 $n!$ 种, n^n 不被 $n!$ 整除,必然不均($n=3$ 时 (1,2,3) 概率 $2/9$)。实证:洗 10 万次统计分布。

章末实验题:0-1 背包与八皇后综合实验

实验 10:0-1 背包与八皇后综合实验

实验目标:综合运用动态规划与回溯两大范式:背包两版实现与验证、八皇后、计时对比。

实验要求:

- 任务 1:实现 knap2d 与 knap1d,均返回最大价值与方案下标(\$10.3.4 二维与一维、\$10.3.3 方案回溯)
- 任务 2:实现八皇后,输出解的个数与**前 3 个解**(\$10.5.2 选择-递归-撤销、冲突检查)
- 任务 3:固定种子随机数据($n=20$, $W=200$)验证两版价值与方案一致(\$10.7.3 随机化测试与对拍)
- 任务 4:计时对比一维与二维($n=1000$ 、 $W=10000$),输出耗时与加速比(\$2.1 计时、\$10.3.4 空间优化)
- 任务 5:输入边界——容量 0、无物品时输出 0 而非崩溃(\$10.3.3 边界与初值)

实验环境:Python 3。运行: `python lab10.py`。

第一步 写 knap2d(二维表 + 回溯选取),再写 knap1d(一维逆序 + pick 表);两版判定相同:dp 值"严格变大"即选中

第二步 皇后复用示例 10-10 骨架:三个标记数组 + 闭包收集;输出时列号加 1 转成 1 基

第三步 机数据用 random.seed(2026),权重 1-50、价值 1-100;同时比较价值与下标序列

第四步 小数据确认正确,再计时(n=1000、W=10000,结果累加防死代码消除);边界调用 knap1d(3,0,...) 与 knap1d(0,10,...) 确认返回 0

✅ 参考答案(完整可运行程序)

```
# lab10.py(第一段:背包两版)
import random
import time

# 任务 1a:二维 DP;返回 {价值, 方案下标}
def knap2d(n, W, w, v):
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for c in range(1, W + 1):
            if c >= w[i - 1]:
                dp[i][c] = max(dp[i - 1][c],
                               dp[i - 1][c - w[i - 1]] + v[i - 1])
            else:
                dp[i][c] = dp[i - 1][c]
    chosen = [] # 回溯选取方案
    i, c = n, W
    while i >= 1:
        if dp[i][c] != dp[i - 1][c]:
            chosen.append(i - 1)
            c -= w[i - 1]
        i -= 1
    chosen.reverse()
    return dp[n][W], chosen

# 任务 1b:一维滚动数组(容量必须逆序!)
def knap1d(n, W, w, v):
    dp = [0] * (W + 1)
    pick = [[False] * (W + 1) for _ in range(n)]
    for i in range(n):
        for c in range(W, w[i] - 1, -1):
            t = dp[c - w[i]] + v[i]
            if t > dp[c]:
                dp[c] = t
                pick[i][c] = True
    chosen = []
    i, c = n - 1, W
    while i >= 0:
        if pick[i][c]:
            chosen.append(i)
            c -= w[i]
        i -= 1
    chosen.reverse()
    return dp[W], chosen
```

lab10.py(第二段:八皇后与主程序)

```

# 任务 2:八皇后(闭包回调)
queens_count = 0
first_sol = []

def n_queens(row, n, pos, col, d1, d2, visit):
    if row == n:
        visit(pos)
        return
    for c in range(n):
        if col[c] or d1[row + c] or d2[row - c + n - 1]:
            continue
        pos[row] = c
        col[c] = d1[row + c] = d2[row - c + n - 1] = True # 选择
        n_queens(row + 1, n, pos, col, d1, d2, visit) # 递归
        col[c] = d1[row + c] = d2[row - c + n - 1] = False # 撤销

if __name__ == "__main__":
    random.seed(2026) # 固定种子,可复现
    # 任务 3:随机数据验证两版
    n, W = 20, 200
    w = [random.randint(1, 50) for _ in range(n)]
    v = [random.randint(1, 100) for _ in range(n)]
    r2 = knap2d(n, W, w, v)
    r1 = knap1d(n, W, w, v)
    print("随机数据 n=%d W=%d 价值: 二维=%d 一维=%d 方案一致: %s"
          % (n, W, r2[0], r1[0], "是" if r2 == r1 else "否"))
    # 任务 4:计时对比
    n, W = 1000, 10000
    w = [random.randint(1, 50) for _ in range(n)]
    v = [random.randint(1, 100) for _ in range(n)]
    t0 = time.perf_counter()
    big2 = knap2d(n, W, w, v) # 结果参与打印,防死代码消除
    ms2 = (time.perf_counter() - t0) * 1000
    t0 = time.perf_counter()
    big1 = knap1d(n, W, w, v)
    ms1 = (time.perf_counter() - t0) * 1000
    print("计时 n=%d W=%d 二维=%.1f ms 一维=%.1f ms 加速比=%.2f"
          % (n, W, ms2, ms1, ms2 / ms1))
    print("大样例价值一致: %s" % ("是" if big2[0] == big1[0] else "否"))
    # 八皇后(前 3 个解 + 总数)
    pos = [0] * 8
    col = [False] * 8
    d1 = [False] * 15
    d2 = [False] * 15

    def visit(sol):
        global queens_count
        queens_count += 1
        if len(first_sol) < 3:
            first_sol.append(sol[:]) # 拷贝,防撤销污染

    n_queens(0, 8, pos, col, d1, d2, visit)
    print("八皇后解的总数: %d" % queens_count)
    for i, sol in enumerate(first_sol, 1):
        print("解 %d: %s" % (i, " ".join(str(x + 1) for x in sol)))

```



```
# 任务 5:边界
wb = [2, 3, 4]
vb = [3, 4, 5]
print("容量 0: %d 无物品: %d" % (knap1d(3, 0, wb, vb)[0],
                                   knap1d(0, 10, wb, vb)[0]))
```

示例输出(本机真实运行;时间因机器而异):

```
随机数据 n=20 W=200 价值: 二维=469 一维=469 方案一致: 是
计时 n=1000 W=10000 二维=1298.0 ms 一维=543.5 ms 加速比=2.39
大样例价值一致: 是
八皇后解的总数: 92
解 1: 1 5 8 6 3 7 2 4
解 2: 1 6 8 3 7 4 2 5
解 3: 1 7 4 6 8 2 5 3
容量 0: 0 无物品: 0
```

设计说明:① 两版共用"dp 值严格变大即选中"的判定,同时比较价值与下标序列,比只比价值更严格;② 一维版用 bool 型 pick 表存选择历史,体积约为二维表的四分之一;计时中一维因缓存友好更快;③ 八皇后用闭包 visit 解耦"搜索"与"结果处理"——正是 §10.5 的闭包模式;④ 固定种子 2026 保证任意机器复现相同数据;⑤ 边界任务验证"初值与边界"检查项:容量 0 与无物品时转移自然失效,输出 0;⑥ 大样例下 Python 二维表的内存与耗时显著高于 C++(对象开销),但两版加速比结构一致,结论不受影响。

下一章预告:第11章 摊还分析——聚集法、会计法、势能法。第3章"list 倍增扩容为何平均 $O(1)$ "的悬念将在那里严格证明;本章的哈夫曼建堆、quickselect 的整体代价也会用摊还视角重新审视。